PROMPT — DESENVOLVIMENTO DO SISTEMA PARKCONTROL

1. Contexto e objetivo

Atue como um engenheiro de software sênior, analista de requisitos, arquiteto de sistemas, desenvolvedor frontend e designer UI/UX.

Sua missão é desenvolver um projeto completo denominado **ParkControl — Sistema de Controle e Gerenciamento de Estacionamento**.

O projeto deverá contemplar duas entregas principais:

1. Um protótipo funcional, moderno, responsivo e interativo, desenvolvido exclusivamente com **HTML5, CSS3 e JavaScript puro**.
2. Uma documentação técnica e acadêmica completa, utilizando arquivos Markdown (`.md`), com requisitos funcionais, requisitos não funcionais, regras de negócio, casos de uso, arquitetura, fluxos e planejamento dos testes.

O sistema deverá simular o funcionamento de um estacionamento real, permitindo controlar veículos, vagas, entradas, saídas, pagamentos e movimentações financeiras.

IMPORTANTE: Não quero apenas imagens, layouts estáticos, exemplos isolados de código ou explicações teóricas. Quero que você crie os arquivos reais do projeto, implemente as funcionalidades e entregue um protótipo navegável e operacional.

---

2. Tecnologias e restrições

Utilize obrigatoriamente:

- HTML5 semântico para a estrutura.
- CSS3 para toda a estilização, incluindo animações e responsividade.
- JavaScript puro (Vanilla JS) para a lógica e interatividade.
- LocalStorage para persistência dos dados do protótipo.
- Markdown (`.md`) para documentação.

Não utilize React, Angular, Vue, Next.js, TypeScript ou frameworks que exijam compilação.

Organize o JavaScript em módulos quando adequado, utilizando módulos ES nativos.

O protótipo deve funcionar localmente, preferencialmente com um servidor estático simples, sem depender de um backend.

Trate a aplicação como um protótipo acadêmico funcional, não como um sistema de produção com segurança ou persistência garantidas.

---

3. Levantamento e especificação dos requisitos

Antes de desenvolver o código, realize uma análise estruturada do sistema e documente seus requisitos.

Crie o arquivo `docs/REQUISITOS.md` contendo:

3.1. Requisitos funcionais (RF)

Especifique pelo menos os seguintes requisitos:

- RF001 — Registrar entrada de veículo.
- RF002 — Registrar saída de veículo.
- RF003 — Consultar veículos atualmente estacionados.
- RF004 — Controlar quantidade de vagas disponíveis e ocupadas.
- RF005 — Identificar veículos pela placa.
- RF006 — Calcular automaticamente o tempo de permanência.
- RF007 — Calcular o valor do estacionamento.
- RF008 — Registrar pagamentos.
- RF009 — Consultar histórico de entradas e saídas.
- RF010 — Pesquisar registros por placa, data e status.
- RF011 — Exibir dashboard com indicadores operacionais.
- RF012 — Configurar capacidade do estacionamento.
- RF013 — Configurar tarifas e tolerâncias.
- RF014 — Gerar comprovante de estacionamento.
- RF015 — Consultar resumo financeiro.
- RF016 — Validar os dados informados pelo operador.
- RF017 — Exportar registros para CSV.
- RF018 — Preservar os dados após atualizar a página.

Você poderá complementar esses requisitos se identificar necessidades importantes.

Para cada requisito funcional, documente:

- Identificador.

- Nome.
- Descrição completa.
- Ator responsável.
- Prioridade (alta, média ou baixa).
- Entradas necessárias.
- Processamento esperado.
- Saídas/resultados.
- Critérios de aceitação.
- Regras de negócio relacionadas.

3.2. Requisitos não funcionais (RNF)

Inclua, no mínimo:

- RNF001 — Interface responsiva.
- RNF002 — Usabilidade e facilidade de navegação.
- RNF003 — Compatibilidade com navegadores modernos.
- RNF004 — Organização e manutenibilidade do código.
- RNF005 — Acessibilidade básica.
- RNF006 — Integridade dos dados.
- RNF007 — Feedback visual das operações.
- RNF008 — Persistência local.
- RNF009 — Desempenho em operações comuns.
- RNF010 — Validação e tratamento de erros.
- RNF011 — Separação entre interface e lógica de negócio.
- RNF012 — Documentação técnica organizada.

Defina critérios verificáveis para cada requisito não funcional, evitando expressões vagas como "ser rápido" ou "ser seguro".

Diferencie os requisitos efetivamente atendidos pelo protótipo das funcionalidades que exigiriam infraestrutura de produção.

---

4. Regras de negócio

Crie o arquivo `docs/REGRAS_NEGOCIO.md`.

Especifique as regras com identificadores RN001, RN002, RN003 etc.

Implemente e documente obrigatoriamente as seguintes regras:

RN001 — Placa única ativa: Um veículo não poderá realizar uma nova entrada enquanto houver uma permanência ativa registrada para a mesma placa.

RN002 — Controle de lotação: O sistema deverá impedir novas entradas quando a capacidade correspondente ao tipo de veículo estiver esgotada.

RN003 — Entrada: Cada entrada deverá gerar um identificador único, placa, tipo de veículo, data e horário.

RN004 — Saída: A saída somente poderá ser realizada para uma permanência ativa.

RN005 — Tempo de permanência: O sistema deverá calcular a diferença entre a data/hora de entrada e a data/hora de saída.

RN006 — Tarifação: O valor será calculado automaticamente de acordo com a permanência e as tarifas cadastradas.

RN007 — Tolerância: O estacionamento deverá possuir um período inicial configurável de tolerância, com valor padrão de 10 minutos.

RN008 — Primeira hora: Ultrapassada a tolerância, será cobrada a primeira hora integral.

RN009 — Frações adicionais: Após a primeira hora, os períodos adicionais serão cobrados em frações iniciadas de 15 minutos, cada uma equivalente a 25% da tarifa horária.

RN010 — Registro financeiro: Somente pagamentos confirmados poderão integrar o faturamento realizado.

RN011 — Vaga liberada: Uma vaga deverá ser liberada somente após a conclusão válida da operação de saída.

RN012 — Histórico: As permanências finalizadas deverão continuar disponíveis no histórico.

RN013 — Validação da placa: Aceitar placas brasileiras nos formatos antigo e Mercosul, normalizando letras para maiúsculas.

RN014 — Integridade: Não permitir valores negativos, saídas anteriores à entrada, capacidades inválidas ou registros incompletos.

RN015 — Capacidade: Impedir que o operador reduza a capacidade configurada para um valor inferior à ocupação atual.

RN016 — Pagamento: Permitir os meios dinheiro, Pix, cartão de débito e cartão de crédito, apenas como registro demonstrativo, sem integração financeira real.

RN017 — Configuração: Alterações nas tarifas deverão valer para novas entradas. A tarifa aplicável a cada permanência deverá ser armazenada no momento de sua entrada, para não haver mudanças retroativas inesperadas.

Utilize valores demonstrativos configuráveis, como R$ 10,00 por hora para carros e R$ 5,00 para motocicletas.

Documente exemplos de cálculo e casos extremos, incluindo:

- Permanência de 5 minutos.
- Permanência de 30 minutos.
- Permanência de 1 hora.
- Permanência de 1 hora e 10 minutos.
- Permanência de 2 horas e 20 minutos.
- Permanência que atravessa a meia-noite.

As regras documentadas deverão corresponder exatamente à implementação em JavaScript.

---

5. Funcionalidades e telas do protótipo

Desenvolva um sistema com navegação lateral (sidebar), cabeçalho e área principal.

Tela 1 — Dashboard

Criar uma página inicial com:

- Total de vagas.
- Vagas disponíveis.
- Vagas ocupadas.
- Taxa de ocupação.
- Quantidade de entradas do dia.
- Quantidade de saídas do dia.
- Faturamento confirmado do dia.
- Lista das movimentações recentes.
- Indicadores visuais de ocupação.
- Gráficos simples de movimentações e faturamento, preferencialmente implementados sem dependências externas.

Os dados devem ser calculados a partir dos registros reais do protótipo, nunca de números fictícios fixos.

Tela 2 — Entrada de veículos

Criar um formulário contendo:

- Placa do veículo.
- Tipo do veículo (carro ou moto).

- Nome do responsável (opcional).
- Data e horário de entrada, preenchidos automaticamente.
- Botão para confirmar entrada.

Realizar as validações necessárias.

Após o registro, atualizar automaticamente a ocupação e os indicadores.

Tela 3 — Veículos estacionados

Exibir uma tabela contendo:

- Identificador.
- Placa.
- Tipo.
- Horário de entrada.
- Tempo de permanência.
- Valor estimado.
- Status.
- Ações.

Permitir pesquisar veículos pela placa, filtrar pelo tipo e realizar a saída.

O tempo e o valor estimado deverão ser atualizados enquanto a permanência estiver ativa.

Tela 4 — Saída e pagamento

Ao selecionar a saída de um veículo:

1. Identificar sua permanência ativa.
2. Exibir data e horário de entrada.
3. Apresentar o tempo de permanência calculado.
4. Apresentar a memória de cálculo da tarifa.
5. Exibir o valor total.
6. Permitir escolher a forma de pagamento.
7. Confirmar o pagamento e a saída.
8. Liberar a vaga.
9. Atualizar os indicadores.
10. Permitir gerar ou imprimir um comprovante.

A operação deverá ser consistente, evitando saídas duplicadas ou registros parcialmente finalizados.

Tela 5 — Histórico

Criar uma tabela de movimentações finalizadas, contendo:

- Placa.
- Tipo de veículo.
- Entrada.
- Saída.
- Tempo total.
- Valor cobrado.
- Método de pagamento.
- Status.

Permitir filtros por data, placa e tipo de veículo.

Adicionar opção para exportar os dados filtrados em CSV.

Tela 6 — Financeiro

Criar uma página com:

- Faturamento total confirmado.
- Faturamento do dia.
- Quantidade de pagamentos.
- Resumo por forma de pagamento.
- Histórico financeiro.
- Filtros por período.
- Gráfico de receitas.

Os valores financeiros deverão ser calculados com base em pagamentos registrados.

Tela 7 — Configurações

Criar formulário para:

- Nome do estacionamento.
- Quantidade de vagas para carros.
- Quantidade de vagas para motos.
- Valor por hora de cada tipo.
- Tempo de tolerância.
- Fração adicional de cobrança.

Persistir as configurações e validar seus valores.

---

6. Design e experiência do usuário

Quero uma interface visualmente profissional, semelhante a um software SaaS moderno.

Utilize como inspiração visual sistemas administrativos, dashboards empresariais e plataformas de gestão.

Diretrizes:

- Layout limpo e moderno.
- Sidebar com ícones e navegação funcional.
- Cards de indicadores.
- Tipografia bem definida.
- Espaçamento consistente.
- Bordas arredondadas.
- Sombras discretas.
- Tabelas organizadas.
- Formulários modernos.
- Modais de confirmação.
- Notificações de sucesso e erro.
- Estados vazios bem desenhados.
- Estados de carregamento apenas quando necessários.
- Responsividade para desktop, tablet e celular.
- Ícones preferencialmente em SVG.
- Estados de foco visíveis e boa navegação por teclado.

Sugestão de paleta:

- Azul-escuro: `#0F172A`
- Azul principal: `#2563EB`
- Verde de sucesso: `#16A34A`
- Vermelho de erro: `#DC2626`
- Cinza de fundo: `#F8FAFC`
- Branco: `#FFFFFF`

Utilize identidade visual consistente com o nome ParkControl.

A interface deverá estar totalmente em português brasileiro, incluindo datas, mensagens e valores em reais.

---

7. Documentação obrigatória em Markdown

Crie uma pasta `docs/` e gere os seguintes documentos:

REQUISITOS.md

Deve conter introdução, contexto, objetivos, escopo, stakeholders, atores, requisitos funcionais, requisitos não funcionais, prioridades e critérios de aceitação.

REGRAS_NEGOCIO.md

Deve conter todas as regras de negócio numeradas, justificativas, exceções e exemplos.

CASOS_DE_USO.md

Documentar os principais casos de uso com:

- Identificador.
- Nome.
- Ator principal.
- Objetivo.
- Pré-condições.
- Fluxo principal.
- Fluxos alternativos.
- Exceções.
- Pós-condições.

Incluir casos como registrar entrada, consultar vagas, registrar saída, realizar pagamento, consultar histórico e alterar tarifas.

HISTORIAS_USUARIO.md

Criar histórias usando o formato:

"Como [ator], quero [funcionalidade], para [benefício]."

Incluir critérios de aceitação objetivos e relacionar as histórias aos requisitos funcionais.

ARQUITETURA.md

Explicar a arquitetura frontend, organização dos módulos, fluxo dos dados, persistência local, modelo das entidades, decisões técnicas e limitações.

Representar visualmente a arquitetura e os fluxos quando possível, usando Mermaid em Markdown.

MODELO_DADOS.md

Definir as entidades e campos necessários.

Considerar entidades como:

- Veículo/permanência.
- Configurações.
- Pagamento.
- Movimentação.

Documentar identificadores, tipos, atributos, relacionamentos lógicos e exemplos de objetos JSON.

`PROTOTIPO.md`

Descrever todas as telas, componentes, navegação, interações, validações e correspondência entre telas e requisitos.

`PLANO_TESTES.md`

Definir testes funcionais, de validação, de integração entre módulos e de interface.

Criar uma matriz de testes com:

- ID do teste.
- Requisito relacionado.
- Cenário.
- Pré-condição.
- Passos.
- Resultado esperado.
- Resultado obtido, se executado.
- Status.

Não registrar testes como aprovados caso não tenham sido executados.

`RASTREABILIDADE.md`

Criar uma matriz relacionando requisitos funcionais, regras de negócio, casos de uso, telas, módulos de implementação e testes.

`README.md` (na raiz)

Documentar:

- Nome e apresentação do sistema.
- Objetivo.

- Funcionalidades.
- Tecnologias.
- Estrutura de pastas.
- Como executar localmente.
- Como utilizar.
- Localização da documentação.
- Limitações do protótipo.
- Próximas melhorias.

Todos os documentos devem ser completos, coerentes entre si e escritos em linguagem acadêmica e técnica clara.

Evite documentação superficial, placeholders ou descrições genéricas.

---

8. Estrutura de arquivos

Organize o projeto aproximadamente da seguinte forma:

parkcontrol/

- index.html
- README.md
- css/
    - style.css
    - components.css
    - responsive.css
- js/
    - app.js
    - storage.js
    - vehicles.js
    - parking.js
    - pricing.js
    - dashboard.js
    - history.js
    - finance.js
    - settings.js
    - utils.js
- assets/
    - icons/
- docs/
    - REQUISITOS.md
    - REGRAS_NEGOCIO.md

    - CASOS_DE_USO.md
    - HISTORIAS_USUARIO.md
    - ARQUITETURA.md
    - MODELO_DADOS.md
    - PROTOTIPO.md
    - PLANO_TESTES.md
    - RASTREABILIDADE.md

Ajuste a estrutura se necessário, evitando arquivos vazios ou módulos desnecessários.

---

9. Persistência e lógica de dados

Como a aplicação não terá backend, utilize LocalStorage para armazenar e recuperar os dados.

Implemente:

- Inicialização segura das configurações padrão.
- Validação dos objetos persistidos.
- Tratamento de dados ausentes ou inválidos.
- Identificadores únicos.
- Separação entre operações de armazenamento e interface.
- Atualização da interface depois das operações.
- Proteção contra duplicidade de entradas e saídas.
- Cálculos financeiros com arredondamento monetário consistente.

Não utilize `eval` nem injete dados fornecidos pelo usuário como HTML não sanitizado.

Deixe explícito na documentação que LocalStorage não é adequado para dados reais sensíveis, operações simultâneas entre vários operadores ou persistência confiável em produção.

Não simule uma autenticação segura apenas com validação no navegador.

---

10. Testes e validação

Crie cenários para verificar:

1. Registro de um carro com vagas disponíveis.
2. Tentativa de registrar duas entradas simultaneamente ativas para a mesma placa.

3. Entrada quando o estacionamento está lotado.
4. Registro de veículo com placa inválida.
5. Cálculo para permanência dentro da tolerância.
6. Cálculo para permanência superior à primeira hora.
7. Cálculo para permanência atravessando dias diferentes.
8. Finalização correta de pagamento e saída.
9. Atualização da quantidade de vagas disponíveis.
10. Preservação dos dados após recarregar a página.
11. Filtros e pesquisas no histórico.
12. Exportação para CSV.
13. Alteração de tarifas e preservação da tarifa de permanências já iniciadas.
14. Validação de configurações inválidas.
15. Consistência dos indicadores financeiros.

Separe as funções de cálculo e validação para permitir testes independentes.

Execute as verificações possíveis no ambiente disponível e relate os resultados reais.

---

11. Critérios de qualidade

O projeto somente estará concluído quando:

- Todas as telas estiverem implementadas.
- A navegação funcionar.
- Os botões possuírem ações reais.
- O cadastro de entradas estiver funcionando.
- O controle de vagas estiver correto.
- O cálculo das tarifas respeitar as regras documentadas.
- O fluxo de saída e pagamento estiver funcional.
- O histórico estiver sendo atualizado.
- O dashboard apresentar valores derivados dos dados.
- A persistência local estiver implementada.
- A interface estiver responsiva.
- Os documentos Markdown estiverem completos.
- Houver coerência entre código, requisitos, regras e testes.
- Não houver erros conhecidos de JavaScript não documentados.

Não crie botões decorativos sem comportamento.

Não utilize dados fictícios permanentes para indicadores operacionais. Se incluir dados de demonstração, ofereça um modo explícito de inicializá-los e restaurar o estado vazio.

---

12. Ordem de execução

Trabalhe em etapas organizadas:

ETAPA 1 — Análise: Defina o escopo, atores, entidades, regras e principais fluxos.

ETAPA 2 — Documentação: Crie os documentos Markdown iniciais com requisitos, regras, histórias, casos de uso, arquitetura e modelo de dados.

ETAPA 3 — Interface: Desenvolva o layout base, componentes, navegação, estilos e responsividade.

ETAPA 4 — Funcionalidades: Implemente entradas, vagas, permanências, tarifação, pagamentos, histórico e configurações.

ETAPA 5 — Integração: Conecte as telas aos dados reais persistidos e atualize dashboard e financeiro.

ETAPA 6 — Testes: Verifique as regras de negócio, fluxos completos, casos de erro e consistência.

ETAPA 7 — Revisão: Atualize a documentação para representar fielmente a implementação final, registre limitações e entregue o projeto.

Não pare apenas na criação dos documentos. Continue até que o protótipo esteja implementado e verificado.

Se surgir alguma ambiguidade não crítica, tome uma decisão técnica razoável, documente-a como premissa e prossiga.

---

13. Entrega final

Ao terminar, apresente um resumo objetivo contendo:

1. Funcionalidades implementadas.
2. Documentos Markdown criados.
3. Estrutura final do projeto.
4. Como executar e testar o sistema.
5. Resultados dos testes realizados.
6. Limitações conhecidas.
7. Melhorias sugeridas para uma versão futura com backend, banco de dados e autenticação real.

O resultado esperado é um sistema de gerenciamento de estacionamento funcional e visualmente profissional, acompanhado de documentação acadêmica completa de Engenharia de Software, com requisitos rastreáveis e regras de negócio efetivamente implementadas.

Comece criando o projeto e seus arquivos, não apenas descrevendo o que pretende desenvolver.

Identificação acadêmica do projeto

Nome do projeto: ParkControl — Sistema de Controle e Gerenciamento de Estacionamento

Desenvolvedores:

- João Pedro Ribas da Silva
- Nicolas Ferreira Souza

Curso: Análise e Desenvolvimento de Sistemas (ADS)

Instituição: Faculdade de Tecnologia de Carapicuíba — FATEC Carapicuíba

Ano: 2026

Requisitos de identificação e autoria

1. Inclua os nomes completos dos dois desenvolvedores no arquivo `README.md`, na seção "Autores".
2. Identifique a instituição e o curso na documentação acadêmica.
3. Crie um arquivo `docs/APRESENTACAO_PROJETO.md`, contendo:
    - Nome do projeto.
    - Identificação dos desenvolvedores.
    - Instituição de ensino.
    - Curso.
    - Ano de desenvolvimento.
    - Contextualização do projeto.
    - Problema identificado.
    - Justificativa.
    - Objetivo geral.
    - Objetivos específicos.
    - Escopo da solução.
    - Tecnologias utilizadas.
    - Resultados esperados.
    - Considerações finais.

4. Inclua uma seção discreta de créditos no rodapé da aplicação com o seguinte texto: **Desenvolvido por João Pedro Ribas da Silva e Nicolas Ferreira Souza | ADS Noite — FATEC Carapicuíba | 2026**
5. Padronize os documentos Markdown com cabeçalho de identificação acadêmica, título, versão e data de atualização.
6. Utilize linguagem formal e acadêmica na documentação, mantendo clareza e objetividade.
7. Não invente orientador, disciplina, semestre ou informações institucionais que não foram fornecidas.

Importante: A autoria deverá ser atribuída igualmente aos dois desenvolvedores em toda a aplicação e documentação. Evite destacar apenas um dos integrantes.